

Agent Harness Engineering: A Survey - 超详细中文研读版

用途说明：这是一份面向课程阅读、组会汇报和导师讨论的中文研读版。它按原论文的章节和小节顺序展开，尽量覆盖原文中的主要论点、例子、代表系统、图表和工程含义，但不是逐字全文翻译，也不复刻原论文版式。

0. 快速总览

这篇论文讨论的是一个正在变得非常重要、但在研究语言中还没有被充分整理的主题：LLM agent 的可靠性到底来自哪里。很多论文和产品宣传容易把注意力放在模型本身，例如模型会不会规划、会不会用工具、会不会写代码、会不会调用 API。但作者认为，在真实生产系统中，一个 agent 能不能稳定完成长程任务，往往更受模型外层基础设施影响。这个外层基础设施就是论文所说的 agent harness。

可以把 agent harness 理解为“把 LLM 包起来让它能干活的工程底座”。它包括执行环境、工具接口、上下文管理、任务生命周期、观测日志、评测验证、权限治理、安全审计、人类接管等机制。模型负责生成下一步动作，但 harness 决定模型看到什么、能调用什么、在哪里执行、失败后怎么恢复、如何记录证据、怎么判断结果是否正确，以及哪些动作必须被禁止或交给人类确认。

论文最重要的观点是 binding-constraint thesis：在长程 agent 任务里，限制可靠性的瓶颈经常不是模型本身，而是 harness。作者引用多个 2026 年左右的结果说明，同一个模型如果换一个更好的工具格式、上下文注入方式、自验证 hook 或自动优化过的 harness，benchmark 成绩可以显著提升；这些提升幅度有时超过一次模型升级带来的提升。

论文提出 ETCLOVG 七层分类法：

- E: Execution Environment and Sandbox, 执行环境与沙箱。
- T: Tool Interface and Protocol, 工具接口与协议。
- C: Context and Memory Management, 上下文与记忆管理。
- L: Lifecycle and Orchestration, 生命周期与编排。
- O: Observability and Operations, 可观测与运维。
- V: Verification and Evaluation, 验证与评测。
- G: Governance and Security, 治理与安全。

这七层不是简单 checklist。前四层是 agent 系统的结构核心，后三层是控制平面。论文强调 O 和 G 应该是独立层，而不是 lifecycle hook 的附属功能，因为在生产系统中，可观测和治理都有自己的工具栈、团队职责和失败模式。

1. Introduction

1.1 The Binding Constraint: Harness over Model

引言首先指出，学术界研究 LLM agent 时，通常把问题中心放在模型能力上：模型能不能多步规划，能不能正确调用工具，能不能检索并压缩记忆，能不能与其他 agent 协作。这个视角隐含一个前提：agent 能力主要是模型能力的函数，模型越强，agent 越可靠。

作者认为这个前提已经被生产经验和近期实验证据挑战。论文举了几类 harness-only gain：只改变编辑工具格式和工具 harness，就能让多个模型在 coding benchmark 上显著提升；固定模型不变，只调整系统提示结构、中间件上下文注入、自验证 hook，就能让 terminal benchmark 成绩上升；自动搜索和优化 harness 甚至可以超过人工设计的方案。这里关键点是模型权重没有变，变的是模型外面的控制器。

这说明 long-horizon agent 不是单个模型调用，而是一个闭环系统。用户目标进入系统后，harness 构造上下文，模型提出动作，工具执行动作，环境返回观察，harness 再把结果注入下一轮上下文，并持续管理状态、权限、评测和恢复。任何一环设计不好，都会让强模型表现得像弱系统。

因此作者提出 binding-constraint thesis：在可比较的前沿模型之间，长程任务 benchmark 的差异可能和执行 harness 一样相关，甚至更多由 harness 决定。论文之后所有章节都围绕这个主张展开。

1.2 The Practitioner-Research Gap

作者接着指出实践界和研究界之间有一个语言缺口。生产团队已经意识到 harness 很重要，但研究界常把相关组件拆开讨论，例如 memory、tool use、planning、safety、benchmark、sandbox，而缺少一个系统工程视角。

OpenAI 将 harness engineering 作为 Codex agent 周围环境、约束、文档和反馈循环的设计 discipline。Anthropic 的多篇工程文章也反复强调：agent 架构要简单可检查；工具接口不能照搬给人用的 API，而要专门为 agent 设计；上下文应渐进式披露，而不是一次性塞满；长任务需要持久 handoff artifact、checkpoint 和可恢复执行环境。

Martin Fowler 网站上的相关文章还把 harness engineering 比作围绕 AI agent 的控制器：前馈指导告诉系统该怎么做，反馈传感器观察结果并纠偏。这个比喻很适合理解本文：agent harness 不是模型旁边的辅助代码，而是闭环控制系统。

1.3 Scope and Contributions

论文的范围是“把语言模型调用变成长程、多步、可控任务执行的基础设施层”。它不系统综述 agent framework 作为开发工具，不讨论 agent platform 作为产品市场，也不评测模型能力本身；它关注这些东西背后的机制。

本文贡献可归纳为三点：

- 概念贡献：提出 harness 是现实 agent 可靠性的独立约束层。作者用近期实验、生产实践、跨层综合和开放问题来支撑这一点。
- 分类贡献：提出 ETCLOVG 七层框架，并把 Observability 和 Governance 从生命周期 hook 中分离出来，作为一等架构层。
- 经验贡献：把 170+ 公开项目映射到这个分类上，观察哪些层生态已经拥挤，哪些层仍然薄弱。

导师汇报时可以强调：这篇论文的价值不在于提出一个新模型，而在于给 agent 系统工程建立了统一词汇表。

2. Background and Taxonomy

2.1 Evolution of Agent Systems

作者把 agent 系统发展脉络分为三个阶段。

第一阶段是 ReAct 时代，大约 2022-2023 年。ReAct 的核心模式是 observe-think-act：模型观察当前状态，进行推理，选择动作，然后环境返回新观察。早期 AutoGPT、BabyAGI 之类系统把模型调用包进任务队列、记忆模块和工具调用表里，展示了自主 agent 的想象力，也暴露了大量基础设施问题：运行失控、上下文爆炸、状态丢失、外部副作用不可控。

第二阶段是工具集成和多 agent 协调，大约 2023-2024 年。Gorilla、ToolLLM、Toolformer 等工作说明工具使用能力可以通过训练或数据诱导，而不只是靠手写 wrapper。CAMEL、ChatDev、MetaGPT、Mixture-of-Agents 等系统探索多 agent 协作，从角色扮演对话到模拟软件组织，再到多模型聚合。同时，SWE-bench、AgentBench、WebArena、GAIA 等 benchmark 让可执行评测基础设施成熟起来。MCP 和 A2A 等协议也开始推动标准化。

第三阶段是 harness turn，大约 2025-2026 年。到这个阶段，模型已经足够强，工程瓶颈转移到运行底座。OpenAI 明确使用 harness engineering 概念，Meta-Harness 证明自动优化 harness 可以超过人工方案，LangChain DeepAgents 通过 harness 改造显著提升 Terminal-Bench 成绩。作者认为，这些趋势标志着 agent 工程进入系统层时代。

2.2 Three Engineering Phases

论文把工程焦点分成 prompt engineering、context engineering、harness engineering。

Prompt engineering 关注单次模型调用的输入文本。工程师优化指令、few-shot 示例、格式约束和 reasoning template。它的对象很窄：一个 prompt、一个模型调用。

Context engineering 关注每一步推理时模型应该看到什么。随着 agent 执行时间变长，问题从“prompt 怎么写”变成“哪些信息应该进入当前上下文”。这包括检索哪些文件、保留哪些历史、压缩哪些工具结果、如何处理上下文窗口饱和、如何避免无关信息污染注意力。

Harness engineering 的范围更大。它设计模型周围的执行控制器：状态如何维护，工具如何暴露，反馈如何注入，权限如何约束，进展如何验证，失败如何恢复。它不是取代前两者，而是包含前两者。一个成熟 agent harness 仍然需要好 prompt 和好 context policy，但它还必须处理执行、观测、评测和治理。

2.3 The ETCLOVG Seven-Layer Taxonomy

ETCLOVG 是论文主体框架。七层可以分成两组。

结构核心包括 E、T、C、L：

- E 决定动作在哪里运行，以及沙箱边界是什么。
- T 决定外部能力如何被描述、发现和调用。
- C 决定模型在短期、中期、长期能看到和记住什么。
- L 决定任务如何跨调用、工具、失败、子任务和交接推进。

控制平面包括 O、V、G：

- O 记录 trace、成本、延迟、失败和可靠性信号。
- V 把任务和 trace 转化成评价、归因和回归反馈。
- G 通过权限、身份、策略、加固、审计和人工监督约束行为。

这套分类的两个关键设计选择是：第一，Observability 独立出来，因为它已经有 Langfuse、Arize Phoenix、OpenLLMetry、OpenTelemetry 等独立生态。第二，Governance 独立出来，因为安全和合规不只是工具调用前的一个 hook，而是包括身份、权限、审计、策略语言、供应链和组织责任。

作者还把 state management 放在 Lifecycle and Orchestration，而不是 Context。因为这里的 state 指 harness 自己继续执行所需的操作状态，例如待办任务、checkpoint、retry、execution status。模型看到的知识和记忆则属于 Context。

2.4 Scope

论文中的 agent harness 比“LLM 周围的所有软件”窄。它特指把模型调用变成边界化、状态化、工具化任务执行的工程 wrapper。一个 agent framework 如果提供状态编排、工具路由、runtime policy hook 或 trace capture，就在范围内。一个薄模型 API wrapper、prompt library、静态数据集、普通容器平台、通用 APM dashboard，如果没有明确 agent-facing 机制，就不在范围内。

2.5 Project Collection Procedure

作者用类似 systematic review 的方式收集项目，来源包括 prior survey 和 benchmark paper、GitHub 搜索、curated project list、包注册表、公司工程博客和 release note。搜索词包括 agent harness、coding agent、LLM agent sandbox、MCP server、agent observability、agent memory、agent evaluation、agent governance 等。

对每个保留项目，作者记录名称、URL、artifact 类型、来源类型、可用状态、年份、GitHub 元数据和用于 ETCLOVG 编码的公开证据。元数据快照定格在 2026 年 5 月左右。

2.6 Inclusion and Exclusion Criteria

纳入条件有三条：公开文档可查；实现或指定了具体 harness 机制；能映射到至少一个 ETCLOVG 层。包括 agent framework、可执行 agent benchmark、agent 执行沙箱、memory/observability/evaluation/governance 系统。

排除对象包括简单 chatbot demo、prompt pack、薄 API client、没有 agent runtime 的静态数据集或 leaderboard、没有 agent-facing 适配的通用基础设施、技术行为无法从公开文档检查的产品页面。

2.7 Coding Protocol

项目按 ETCLOVG 多标签编码。primary layer 是项目最核心机制，secondary layer 只有在文档明确展示独立能力时才分配。作者使用 single-primary-coder 加 author audit，而不是正式多标注者一致性研究，因此没有报告 Cohen's kappa。模糊案例采用保守规则：没有明确公开证据，就不分配对应层。

2.8 Limitations of the Corpus

语料不是完整 census，而是可见生态图谱。偏差包括英文来源、GitHub 可见项目、开源项目、愿意公开实现细节的系统。商业生产系统可能低估，coding agent 可能高估，因为 coding agent 有丰富公开 artifact：仓库、benchmark、沙箱、issue-to-PR workflow、release note。

2.9 Aggregate Analysis

总体结果是生态 broad but uneven。Execution、Tooling、Lifecycle、Verification 最密，因为 coding/web/terminal/computer-use agent 必须有可执行环境、工具契约、控制循环和可重复评测。Context and memory 很常见，但常嵌在大框架中而不是独立 harness 组件。Observability 和 Governance 在开源中较薄，更多存在于商业平台、SDK 或工程文章。

3. Execution Environment and Sandbox (E)

3.1 Scope and Concepts

执行环境是 agent 动作真正发生的基础设施层。对 LLM agent，执行环境和沙箱几乎不可分。因为 agent 可能执行 shell、读写文件、运行代码、安装依赖、访问网络、操作浏览器或桌面应用。没有沙箱，任何错误或恶意指令都可能直接影响宿主环境。

3.1.1 Definition

执行环境强调“动作物理上在哪里执行”。沙箱强调“这个执行环境被什么边界限制”。在 agent 场景下，两者共同构成 E 层。

3.1.2 Why Sandboxing Is Central in the Agent Era

沙箱在 agent 时代有三重意义。

第一是安全。LLM 生成的命令很难规模化人工审计；agent 会多步自主行动，执行时不一定有人工介入；prompt injection 可能把一个正常任务劫持成攻击任务。传统沙箱保护多租户代码，agent 沙箱还要保护“被语言输入操控的行动者”。

第二是可复现。benchmark、训练和回归评测要求环境能重置到已知状态。Docker container、microVM、浏览器 profile、桌面 VM 都可以销毁并重建，而开发者本机不能。没有 cheap reset，就很难做长程 agent 评测。

第三是活性。没有沙箱时，每个风险动作都要问用户，用户要么烦了放弃，要么习惯性全批准，安全机制失效。沙箱把“每次动作授权”改成“会话边界配置”：边界内 agent 可以自由执行，边界外再审批。这个“既是笼子又是许可证”的双重角色，是 agent 沙箱区别于传统沙箱的关键。

3.2 Categories of Agent Sandboxes

作者按 workload/use case 把沙箱分七类，而不是按容器、microVM、gVisor、WASM 等底层隔离技术分类。因为同一种隔离技术可以服务不同任务。

3.2.1 General-Purpose Managed Sandboxes

通用托管沙箱提供 sandbox-as-a-service，允许通过 API 启动任意 OCI image，并暴露 shell、文件系统、网络、解释器等能力。代表系统包括 Daytona、E2B、Modal、Northflank、OpenSandbox、Docker Sandboxes。

这一类的共同设计是：默认短生命周期，可选持久 session；用 Python/TypeScript SDK 暴露接口；支持任意容器镜像。差异在隔离强度和运维模型。E2B、Modal、Docker Sandboxes 等倾向 microVM 或 gVisor；Northflank 可选择多个后端。趋势是从共享内核容器向 dedicated-kernel microVM 迁移，因为 LLM 生成代码的 syscall 模式很难提前预测。

3.2.2 Computer-Use Agent Infrastructure

Computer-use agent 不通过 API 或 shell，而是看屏幕、点鼠标、敲键盘。代表包括 Anthropic Computer Use、CUA、OSWorld。它们通常封装完整桌面环境或 VM，暴露像素坐标、截图和键盘鼠标动作。

这种方式保真度高，能操作没有 API 的应用；但启动慢、资源重、动作空间巨大，对视觉 grounding 要求高，安全面也更大。因此更适合 microVM/full VM，而不是轻量容器。

3.2.3 Code-Specialized Sandboxes

代码专用沙箱针对代码生成、数据分析和评测优化，代表包括 Judge0、OpenAI Code Interpreter、sandboxed.sh、langchain-sandbox。它们通常预装解释器、编译器和数据分析库，倾向 stateless request-level execution，重点是并发、启动速度和评测吞吐。

一个趋势是从容器沙箱转向 WebAssembly 沙箱。WASM 具有 capability-based security、确定性执行和极快实例化优势，但标准库和 native extension 支持有限。

3.2.4 Framework-Integrated Runtimes

框架内置 runtime 是 agent framework 自带的执行环境，例如 OpenHands runtime、agent-infra sandbox、smolagents executor layer。这些 runtime 通常与框架的编排 loop、工具注册、prompt 约定绑定。

优点是开箱即用，浏览器、shell、文件系统、API server、VSCode/MCP 等能力打包完整。缺点是镜像大、启动慢、与框架强耦合。这里的核心权衡是 bundle vs compose：是把能力预先打包进一个大环境，还是运行时按标准协议组合小组件。

3.2.5 Browser Evaluation Environments

浏览器评测环境既是沙箱也是评测 harness。WebArena、VisualWebArena、BrowserGym、WorkArena 提供隔离 Web 应用、浏览器操作接口和可复现任务定义。它们连接 E 层与 V 层：同一个系统既定义 agent 运行环境，也定义 benchmark 成功条件。

浏览器环境还是研究 indirect prompt injection 的天然场所，因为网页内容本身不可信，agent 会把网页文本、视觉元素、DOM 状态作为行动依据。

3.2.6 OS-Level Permission Sandboxes

OS 级权限沙箱不创建完整新环境，而是用 bubblewrap、Seatbelt、seccomp-bpf 等 OS 机制限制本机文件系统和网络权限。代表包括 Anthropic sandbox-runtime、Claude Code sandboxing、IsolateGPT。

这类系统的哲学是 permission, not partition。目标不是每次给 agent 一个全新 OS，而是在用户本机上给 agent 一个收窄视图。它适合防 prompt-injected command 修改敏感文件或外传数据，但不适合抵御完全恶意代码，因为仍共享宿主内核。

3.2.7 Sandbox Abstraction Layers

沙箱抽象层不是沙箱本身，而是统一多个沙箱后端的 API。SWE-ReX 把 Docker、AWS Fargate、Modal、Daytona 包成可替换 runtime；smolagents executor interface 暴露 local/e2b/modal/docker/wasm 等参数化后端；Kubernetes Agent Sandbox 用 CRD 和 controller 管理 gVisor/Kata 后端。

这一类说明执行 substrate 正在变成可替换组件。agent 编排逻辑不应该硬编码某个 vendor API。抽象层降低迁移成本，也让不同威胁模型下选择不同 sandbox 成为可能。

3.3 Threat Model and Sandbox Escape

agent 沙箱要面对传统威胁：容器逃逸、侧信道、资源耗尽。还要面对三类 agent-specific 风险。

第一，prompt injection 可以让外部输入诱导 agent 发起恶意沙箱操作。第二，目标错位可能让 agent 把突破约束当成完成任务的手段。第三，多工具组合会放大单个弱点，一个 sandbox 漏洞可能通过网络、文件、凭证、浏览器等能力级联。

论文提到 SandboxEscapeBench 对前沿模型在 nested sandbox CTF 场景下进行评测，发现 Docker 配置下存在实际逃逸成功率。防御方面，IsolateGPT、transactional sandboxing、LLM-in-Sandbox 等提出隔离、回滚、最小能力环境等方向，但整体仍零散。作者认为还缺少统一 agent-native runtime security framework。

3.4 Deployment Modes

沙箱部署有三种模式。

自托管模式由开发者管理 sandbox，OpenHands、SWE-agent 常见。优点是延迟低、可控性强，缺点是运维和安全责任重。

云 SaaS 模式由 E2B、Modal、Daytona Cloud 等提供托管执行，优点是弹性扩容和托管安全，缺点是网络往返、数据边界和成本。

混合/BYOC 模式把 agent 逻辑和执行环境解耦，例如本地保留敏感数据，云端提供执行容量。合规、审计、数据驻留要求会推动企业采用混合模式。

3.5 Summary

E 层的核心总结是：沙箱提供安全边界、复现机制和长程自主执行的许可区域。不同沙箱设计取决于任务类型、威胁模型、启动延迟、隔离强度、成本和部署模式。执行层问题会在后面反复出现，因为它影响评测、治理、成本和编排。

4. Tool Interface and Protocol Layer (T)

工具层定义 agent 如何发现能力、描述可调用动作、跨运行边界执行动作。它位于能力扩展和可靠性控制的冲突点：工具越多，agent 能做的事越多；但动作空间越大，选择错误、token 成本、prompt injection 面和规划难度越高。

4.1 Protocol and Interface Standards

MCP 是当前最重要的工具/上下文集成协议之一。它采用 host-client-server 架构，基于 JSON-RPC 交换 tools、resources、prompts。MCP 的价值不仅是 schema 互操作，更是生态流动性：agent 构建者可以复用 MCP server catalog，而不是为每个部署手写 connector。

A2A 面向另一个边界：agent 应用之间的通信。它关注 Agent Card 发现、同步/流式交互、长程任务协作。MCP 更像 agent 到外部能力，A2A 更像 agent 到 agent。

Function calling 和 OpenAPI 仍是基础。Function calling 用 JSON schema 明确工具调用格式；OpenAPI 用机器可读规范描述 HTTP API，供框架生成工具定义和验证器。AGENTS.md/AGENT.md 让仓库直接携带 agent 工作约束和工具使用说明。

论文建议按 integration boundary 分类标准：

- Model 到 Function：结构化调用。
- Agent 到 Capability：运行时能力解耦。
- Agent 到 Agent：跨 agent 委托与协作。
- Agent 到 Repo/Environment：仓库和环境级策略。

4.2 Tool Description, Discovery, and Selection

协议解决“怎么调用”，但更难的问题是“当前该给模型看哪些工具”。如果把所有工具都塞进 prompt，模型可能被无关工具干扰，token 成本也会暴涨。

EASYTOOL、AnyTool、CRAFT、MetaTool、MCP-Zero、ToolRet、ToolRegistry 等工作从不同角度处理工具文档质量、工具检索、动态候选裁剪和 registry 质量。SkillRouter、SkillRet 将类似问题扩展到 skill library：agent 不只是选 API，还要选合适的程序化技能模块。

工程原则有两条。第一，fewer but better tools 往往优于大而全工具菜单。第二，工具发现应动态，而不是全局静态列表。仓库、企业租户、权限状态和任务上下文都会改变“应该暴露哪些能力”。

4.3 Tool-Augmented Training and Integration

工具能力也可以通过训练获得。Toolformer 展示模型可自监督学习何时插入 API 调用；Gorilla、ToolLLM/ToolBench 用更大工具语料和指令微调提升 API 使用；ToolkenGPT、CREATOR 关注调用格式和控制器集成。

生产 harness 通常还要结合框架层 runtime，如 LangChain、Semantic Kernel、smolagents。coding agent 还需要更语义化的工具，如静态分析、类型检查、求解器、证明助手、patch equivalence checker、fault localization checker。

作者特别强调，推理/验证工具不应只返回黑盒 yes/no，而应返回证据型 artifact：trace、proof obligation、counterexample、structured certificate。这样后续 O/V/G 层才能观测、评测和审计。

4.4 Scalability and Session Management

长程工具使用会带来 session 管理问题。Stateful tool session 有助于连续性，但会造成 stale handle、重试不一致、并行调用状态冲突、verbose trace 挤占上下文等问题。

harness 需要显式管理 tool session 生命周期：何时创建、何时复用、何时关闭、失败如何恢复、哪些工具结果进入上下文、哪些只进入 trace。可观测 hook 也必须能区分 planner 错误和协议/接口错误。

5. Context and Memory Management (C)

Context and Memory 是 prompt engineering 和 harness engineering 接合最紧密的一层。它决定模型每一步看到什么，以及知识如何跨轮次、跨会话、跨 agent 实例持久化。

5.1 Why Context Must Be Engineered

作者首先反驳“上下文窗口更大就够了”的直觉。

第一，attention 成本近似二次增长。输入 token 翻倍，注意力计算和显存不是线性增长，而是更快增长。FlashAttention、位置编码扩展等技术能降低常数，但不改变 context 作为稀缺资源的事实。

第二，Lost in the Middle 说明信息位置很重要。相关文档在上下文开头或结尾时更容易被模型使用，放在中间准确率明显下降。这意味着检索到正确内容还不够，放错位置仍然可能失败。

第三，context rot 说明输入变长会让模型退化，而且退化可能在窗口远未填满时就出现。语义匹配弱、需要推理定位的信息更容易受影响。对 agent 来说，工具结果、历史轨迹、文件内容不断堆积，context rot 是常态。

结论：上下文管理不是优化项，而是可靠性核心。包括什么、放哪里、何时删、如何压缩，都直接影响行为。

5.2 From Prompt Engineering to Context Engineering

Context engineering 目标是管理“推理时全部 token 状态”，而不只是 prompt 文本。一个部署中的 agent context 包含系统提示、行为规则、工具定义、历史消息、工具结果、检索文档、长期记忆、动态工作状态。

核心原则是：每一步给模型最小且高信号的信息集合。Progressive disclosure 只在需要时加载信息；compaction 删除已经消化的冗余轨迹；memory retrieval 只取当前相关记录；tool schema 只暴露可用工具。

作者把 memory 分成三层：短期活动上下文窗口、中期 session/cross-run state、长期持久 memory。这类似操作系统内存层次：寄存器/RAM/磁盘各有不同延迟、容量和访问策略。

5.3 Short-Term: Managing the Active Context Window

短期上下文是最直接影响下一步输出的内容。

系统提示校准需要控制“海拔高度”。太具体会脆弱、维护困难；太抽象会缺少行动指导。有效 prompt 往往分成背景、指令、工具指导、输出格式等清晰区块。推荐流程是从最小 prompt 和最强模型开始，观察失败，再针对具体失败补充约束。

工具定义是主要 token 消耗者。每个工具名、描述、参数和 schema 都可能每轮注入。生产原则是工具少而表达力强，参数命名清楚，错误处理鲁棒，工具用途无歧义。如果人类工程师都难以判断何时用哪个工具，模型也很难可靠选择。

Just-in-time retrieval 和 progressive disclosure 让 agent 先保留路径、query、链接、索引、摘要，需要时再加载全文。Claude Code 一类系统通过项目指导文件提供背景，通过 grep/glob 等工具按需探索文件。这比一次性塞入全仓库更可靠。

KV-cache-aware context design 是生产成本优化核心。prompt prefix 稳定可以复用缓存；append-only 历史避免打破前缀；确定性 JSON 序列化避免同一内容因键顺序不同导致缓存 miss。工具列表放在前缀附近，因此运行时频繁增删工具会破坏缓存。Manus 通过状态机 mask 不可用动作，而不是改变工具定义列表，来保持缓存稳定。

5.4 Mid-Term: Session State and Cross-Run Persistence

中期上下文解决一次窗口和长期记忆之间的空档。它让 agent 在 context reset、会话中断或下一次运行时恢复关键状态。

结构化笔记是最简单方法：agent 维护 NOTES.md、todo.md、progress.md，记录地图、目标、决策、未解决问题和下一步。Anthropic 的示例中，agent 在长期游戏过程中通过自写笔记跨越上下文清理。

文件化计划把任务状态、spec、依赖图、中间结果写到文件系统。这样并非所有历史都要进入模型上下文，只有当前步骤相关部分被加载。

跨运行注入把上一轮 session 的关键总结注入下一次启动。优点是无需向量库或图数据库，成本低，部署简单。缺点是历史越来越长时不可扩展，也无法精确检索很久以前某条观察。

5.5 Long-Term: Persistent Memory Systems

长期记忆提供 indexed, retrievable storage，跨 session、任务和 agent 实例持续存在。关键能力是 arbitrary recall：给定 query，找出相关记忆，而不是只把上次摘要往前传。

5.5.1 Foundational Architectures

MemGPT 把 LLM 上下文窗口类比成 RAM，把外部存储类比成 disk，让模型通过函数调用 page in/page out。这个操作系统隐喻非常重要：agent 不是拥有无限上下文，而是通过 memory manager 获得“仿佛有更大记忆”的能力。

Generative Agents 的 MemoryStream 架构将所有观察以自然语言记录存储，带时间戳和重要性分数。检索时结合 recency、importance、relevance。Reflection 机制定期把观察综合为更高层洞见。Observation、reflection、retrieval 三件套成为后续 agent memory 的标准模板。

MemoryBank 加入动态遗忘和用户画像。记忆会随时间衰减，频繁访问会增强，矛盾记忆需要处理。它还维护层次化用户 personality summary。

5.5.2 Production Memory Systems

Mem0 是生产长期记忆代表，组合向量库、图数据库和 KV store。向量库负责语义相似，图数据库负责关系，KV 负责快速事实。LLM extraction layer 从交互中提取事实和偏好，并路由到不同存储。

A-MEM 借鉴 Zettelkasten，不把记忆当平面记录，而是每条记忆带关键词、标签、上下文描述和动态链接。新记忆加入时，还会更新相关旧记忆的上下文和属性。

Hindsight 强调“学习”而不仅是“记住”。它把 retain、recall、reflect 作为 API，生成基于证据的知识记录。

Honcho 更像用户模型系统，用后台 reasoning pipeline 提炼用户偏好、沟通风格和目标变化。多 agent 场景中，每个 agent 保持自己的观察角度，同时共享用户理解。

Mozilla cq 关注 collective memory。多个 agent 不应重复踩同样坑，组织层记忆库可以让一个 agent 失败后的经验被其他 agent 使用。

5.5.3 Academic Surveys and Taxonomy

记忆综述通常把 memory 分成短期 working memory 和长期 semantic/procedural/episodic memory。write-read-manage loop 成为描述记忆系统行为的标准词汇。更高阶框架还提出 monolithic context、context + retrieval store、tiered memory with learned control policies 三类模式。

5.6 Long-Horizon Techniques: Keeping Agents Coherent Over 100+ Turns

长程任务需要同时协调三层 memory。

Context compaction 在窗口接近上限时生成压缩状态并重启执行。难点是 recall/precision 权衡：一开始应最大化保留，避免丢掉未来需要的信息；之后再删冗余。工具结果清理是轻量 compaction：把完整输出替换成路径或引用。

Sub-agent context isolation 让子任务在独立窗口中探索，最终只返回 condensed summary 给 orchestrator。这避免大量局部探索污染主上下文。代价是共享上下文会破坏 KV cache，并增加 prefilling 成本。

Hybrid decision framework 根据任务结构决定策略：总是需要的信息预加载；条件需要的信息即时检索；窗口饱和时压缩；探索型子任务交给 sub-agent；失败时 checkpoint/resume。

5.7 Context Drift and Limits

Context drift 是长程轨迹中 agent 行为逐渐偏离初始意图或真实任务状态。它不同于 context rot。Context rot 是单次输入长导致性能退化；context drift 是多轮压缩、检索、总结、外部化过程中误差累积。

当前技术都有限。Compaction 可能丢掉细节；retrieval 依赖 agent 提出正确 query；sub-agent isolation 只能隔离子任务污染，不能阻止 orchestrator 自身漂移。作者认为，需要 observability、verification、governance、人类 checkpoint 和 anomaly detection 共同解决。

6. Lifecycle and Orchestration (L)

Lifecycle and Orchestration 管理 agent 如何跨模型调用、工具调用、失败、修订和 handoff 推进任务。它把控制流和控制流读写的操作状态放在同一层。

6.1 Lifecycle State Management

生命周期状态包括 pending subtasks、tool results、intermediate artifacts、repo changes、coordination metadata、session persistence、checkpoint/resume。它让 agent run 可持续，而不是每一步都像孤立模型调用。

无状态 replay 通过历史交互重建执行，优点是可复现、可审计；缺点是轨迹长时成本高。有状态执行把状态存在文件、数据库、仓库、任务图或服务里，优点是连续性好；缺点是需要处理一致性和调试复杂度。实践中常见混合模式：既保留 replayable history，又依赖持久 artifact。

这一层的 state 与 C 层 memory 不同。C 层是给模型看的信息；L 层是 harness 自己调度和恢复所需的操作状态。

6.2 Single-Agent Inner Loop

单 agent 内循环是最基本执行单元。一个 agent 反复观察环境、推理、调用工具、接收反馈、继续下一步。Codex CLI、Claude Code、Gemini CLI、OpenCode、Aider、SWE-agent 等都以这种 loop 为基础。

即使是单 loop, harness 也决定很多行为：如何构造 prompt, 哪些工具可见, 工具结果如何压缩, 何时停止, 错误如何重试, 哪些动作需要授权。长程单 agent 容易遇到上下文碎片、累积错误、任务分解弱、过早宣布完成等问题, 因此推动更结构化编排。

6.3 Multi-Agent Orchestration Patterns

多 agent 编排将计划、执行、检查、聚合等职责拆开。常见模式包括：

- 层级编排：上层 controller 分配任务, 下层 agent 执行。
- 团队编排：多个角色 agent 协作, 如 planner、coder、reviewer。
- Workflow 编排：把 agent 和工具组织成显式阶段。
- Fan-out：并行探索多个方案, 再聚合。
- Graph composition：把 agent、tool、state 表示为图节点, 支持复杂控制流。

代表系统包括 AutoGen、LangGraph、Semantic Kernel、OpenAI Agents SDK、DeepAgents、DeerFlow、Hive、Emdash。多 agent 通常需要有状态或混合执行, 因为要维护角色、任务图、共享 artifact 和协调状态。它提高分解和并行能力, 但引入通信、同步和责任归因成本。

6.4 Full Lifecycle Pipeline from Issue to Pull Request

全生命周期编排把 agent 放入完整 workflow。例如从 GitHub issue 开始, 经过需求理解、计划、代码修改、测试、review、PR。Vibe Kanban、Symphony、GitHub Agentic Workflows 代表这一类。

task runner 是核心抽象：负责调度、状态持久化、重试、验证、迭代 refinement。这里 human 不再每一步写代码, 而是 steer、review、approve；agent 执行具体工作。

7. Observability and Operations (O)

可观测性回答“agent 到底做了什么, 为什么失败, 成本在哪里, 如何恢复”。作者把它作为独立层, 因为生产 agent 需要专门工具链, 而不仅是打印日志。

7.1 Tracing and Monitoring Platforms

基础是结构化 trace collection。每次 LLM call、tool invocation、retrieval step、context assembly 都记录为 span tree。Langfuse、Opik、Arize Phoenix、MLflow 等提供 trace tree、latency flamechart、token usage、cost attribution、prompt versioning。

OpenTelemetry 成为底层 instrumentation 标准。GenAI semantic convention 定义模型名、temperature、token count、latency 等 span 属性。OpenLLMetry、OpenInference 将 OpenAI、Anthropic、Cohere、vector DB 等自动接入 OTel, 使 agent trace 可流入 Prometheus、Jaeger、Grafana、Datadog 等常规运维系统。

更底层的 AgentSight 用 eBPF 在应用进程外监控 TLS 边界和内核事件, 将 LLM 意图与进程创建、文件 I/O、网络调用关联。AgentTrace 设计统一 schema, 覆盖认知 reasoning、操作 tool call、上下文环境三类 surface。

7.2 Agent-Specific Operations Platforms

通用 trace 平台关注 LLM call 和 tool call, agent-specific ops 还关注 session、agent identity、role、task、workflow、handoff。AgentOps SDK 提供按 agent lifecycle 组织的层级 span。RagaAI Catalyst 面向 RAG 和多 agent anomaly。Laminar 结合 trace、evaluation、prompt management。

学术工作将 AgentOps 自动化拆成 observe、collect metrics、detect anomalies、root-cause analysis、recommend fixes、automate remediation。另一个方向是 cognitive observability：不仅看 agent 做了什么，还试图解释为什么这样做。Watson 用 surrogate agent 复现输出并生成 reasoning trace，AgentLens 用可视分析呈现轨迹、事件栈和环境回放。

7.3 Cost Tracking and Optimization

agent 成本比单次 LLM call 更复杂，因为一个用户任务可能触发几十次模型调用、工具结果注入、检索和重试。observability 要同时做 tracking 和 optimization。

tracking 方面，TensorZero 将 gateway、observability、experimentation、optimization 结合；Helicone 作为 drop-in proxy 提供成本和延迟监控。

optimization 方面，FrugalGPT 提出 prompt adaptation、LLM approximation、LLM cascading，用便宜模型处理简单问题，贵模型处理困难问题。GPTCache 用语义缓存减少重复调用。QC-Opt 联合优化模型选择、token count 和 latency。token elasticity 提醒我们：过低 token budget 可能反而让 reasoning 溢出并增加成本。Dual-Pool Token-Budget Routing 从服务层按短/长上下文池路由请求。

作者强调成本可观测必须跨层：API token、应用路由、KV cache、GPU 资源、工具调用次数、重试次数都要记录。否则成本优化可能损害评测真实性。

7.4 Reliability Engineering

长程 agent 需要故障恢复、checkpoint/resume、retry、session recovery。Anthropic 总结了 coding agent 常见失败：想一次性完成整个任务；过早宣布完成；会话之间留下坏环境；没测试就标记完成。

工程解法是 harness 设计而不是只换模型：initializer agent 先拆任务，建立 feature list、git repo、progress file、init script；coding agent 逐 feature 工作并留下干净 handoff state。planner-generator-evaluator 三 agent 架构进一步分离计划、生成和评估，避免 agent 自评过度乐观。

Managed Agents 架构把 brain、hands、session 分离。brain 是 harness + LLM，hands 是 sandbox 和 tools，session 是 durable event log。如果 harness 崩溃，可用 sessionId 恢复；如果 sandbox 崩溃，可重新 provision；凭证存 vault，不进入 sandbox。

学术分类包括多 agent 失败、memory/reflection/planning/action/system 模块错误、error propagation、version drift、成本驱动性能塌陷、认知退化等。检测策略应分三层：规则检查格式和 schema；统计监控延迟/成本/成功率 drift；LLM 或人工分析 reasoning failure。

7.5 Toward Unified Observability

论文指出 observability 和 evaluation 常脱节。很多团队收 trace，但不把 trace 系统化变成 eval。理想状态是：生产异常 trace 自动生成 regression test；在线评价分数进入告警；评测失败反过来指导 prompt/tool/context/orchestration 改动。

作者还提出 harness-as-assumption principle：每个 harness 组件都隐含“模型自己做不好”的假设。随着模型升级，可观测系统不仅要发现失败，也要发现哪些组件已经不再必要。

8. Verification and Evaluation (V)

评测层将 agent 行为转化为工程证据。作者强调，agent benchmark 分数是 model-harness pair 的属性。比较模型时必须固定 harness；比较 harness 时必须把 harness config 当实验变量。

8.1 Harness Evaluation as a Task-to-Feedback Lifecycle

传统 LLM eval 通常是固定输入、评分输出。agent eval 是执行 episode：任务嵌入环境，agent 调工具、改状态、处理错误，最终被评测。评测不应只报告 final score，还要产出 judgement、failure attribution 和 regression feedback。

五阶段 lifecycle：

1. Task and Benchmark Grounding。
2. Pre-execution Readiness Validation。
3. Controlled Execution and Trace Capture。
4. Multi-level Judgement and Failure Attribution。
5. Continuous Regression and Deployment Feedback。

8.2 Stage 1: Task and Benchmark Grounding

agent 任务需要明确定义环境状态、可用工具、允许动作、约束、终止条件、成功标准。没有 grounding，分数不可解释。

8.2.1 Software Engineering and Terminal Tasks

SWE-bench 将任务绑定真实 GitHub issue 和 repo snapshot，通过测试判断 patch 是否解决问题。Terminal-Bench 让 agent 在终端环境编辑文件、运行命令、安装依赖、理解失败。强 outcome verifier 依赖强 grounding：仓库状态、依赖、测试和成功条件都要明确。

8.2.2 Web, Browser, and Computer-Use Tasks

WebArena、VisualWebArena、BrowserGym、WorkArena、OSWorld 将任务 grounded 在浏览器、桌面或应用状态。它们说明评测和环境设计不可分。benchmark 必须提供真实接口、隔离任务状态、合适 observation、可测成功谓词。

8.2.3 Cross-Domain and Enterprise Workflow Tasks

AgentBench、GAIA、The Agent Company、WorkArena++ 测试跨工具、跨环境、跨工作流能力。它们的贡献是覆盖度：看 harness 能否在不同工具接口、状态表示和成功条件下泛化。

8.3 Stage 2: Pre-execution Readiness Validation

执行前必须检查环境、工具、上下文、权限、预算、grader 是否正确初始化。否则失败归因会混乱。

8.3.1 Environment and Sandbox Readiness

环境 readiness 包括 repo snapshot、依赖、测试可执行性、浏览器 profile、桌面状态、服务可用性、网站 reset。Repo2Run、HAL 等关注自动化可执行环境和标准化评测基础设施。环境本身就是测量仪器的一部分。

8.3.2 Tool, Context, and Permission Readiness

Tool readiness 检查 API、MCP server、browser control、shell command、file operation 是否可用且描述一致。

Context readiness 检查 history、memory、scratchpad、retrieved docs 是否 reset 或有意初始化。Permission readiness 检查文件、凭证、网络、approval gate 是否符合 benchmark spec。

8.3.3 Evaluator and Grader Readiness

grader 也必须被验证。确定性 grader 要检查 flaky test、依赖、环境兼容；LLM-as-Judge 要版本化 prompt、rubric、judge model；人工审计协议要提前定义。Evaluator 不是系统外的神谕，而是需要测试的组件。

8.4 Stage 3: Controlled Execution and Trace Capture

Rollout 是 agent eval 基本单位，包含任务、模型配置、harness 配置、动作序列、中间观察、最终状态和评分。受控 rollout 固定环境状态、工具、timeout、budget、permission policy、evaluator version。非确定性仍存在时，应多次 rollout 暴露方差。

8.4.1 Controlled Rollouts and Reproducible Execution

SWE-agent、OpenHands、SWE-ReX、LangChain evaluation patterns 都说明，agent eval 复现的不只是模型调用，而是完整 model-tool-environment interaction。

8.4.2 Trace-Native Evaluation

Trace-native harness 应记录模型输出、工具调用、工具结果、状态变化、上下文快照、错误、重试、恢复动作、token、latency、cost。trace 能区分类似失败：是没找到文件、找到了但 patch 错、工具描述误导、还是测试环境坏。

8.4.3 Cost, Latency, and Resource Tracking

长程 agent 可能用更多 token、更多工具调用、更强模型和更多重试换成功率。因此 eval 应报告 success-cost-latency frontier，而不是单一成功率。部署场景中，真正优化对象是预算和延迟约束下的重复 workload。

8.5 Stage 4: Multi-level Judgement and Failure Attribution

Stage 4 包括判断和归因。判断分三层：outcome、trajectory、evaluator。

8.5.1 Outcome-level Evaluation

Outcome-level 判断最终目标是否达成。软件工程用测试，终端任务用文件或命令输出，浏览器任务用最终页面/系统状态，GAIA 等用答案字符串或结构化响应。优点是可规模化、容易比较；缺点是压缩太多，无法说明路径是否安全、鲁棒、高效。

8.5.2 Trajectory-level Evaluation

Trajectory-level 判断 agent 过程质量：工具是否选对，动作顺序是否合理，是否重复无用调用，是否遵守权限，是否从错误中恢复，是否保持上下文一致。最终答案正确但过程违规或极度浪费，也应被识别。

这一层对 harness engineering 特别重要，因为它能指出该改哪层：选错工具可能是 T 层；忘记约束可能是 C 层；循环不止可能是 L 层；权限违规可能是 G 层。

8.5.3 Evaluator-level Evaluation

Evaluator-level 评估 grader 是否可信。确定性 verifiers 稳定、便宜，但覆盖窄；LLM-as-Judge 灵活，但有偏差、不确定和成本；人工审计适合高风险和模糊案例，但不能大规模持续运行。可靠 harness 应采用 layered graders：客观状态用确定性检查，语义和轨迹用 LLM judge，歧义和安全关键点用人工。

8.5.4 Failure Attribution Across Harness Layers

失败可能来自模型推理、工具接口、上下文压缩、沙箱依赖、编排 loop、benchmark 歧义或 evaluator 不稳定。归因应是基于 full trace 的诊断过程，而不是给最终失败贴一个单标签。

8.6 Stage 5: Continuous Regression and Deployment Feedback

agent harness 持续变化：prompt、tool schema、MCP server、context policy、sandbox image、permission rule、judge prompt 都会改。因此 regression eval 应由 harness 变更触发。局部改进可能全局回归。

8.6.1 Regression Evaluation for Harness Changes

推荐维护分层评测套件：tool schema 和 deterministic validator 的 unit-like test；单步决策测试；完整 rollout 测试；长程 coherence simulation。

8.6.2 Evaluation Frameworks and LLMOps Integration

Promptfoo、DeepEval、RAGAS、lm-evaluation-harness 等提供可复用评测基础设施。虽然不一定原生面向长程 agent，但可以作为 regression harness 组件。更重要的是让 production trace 和 evaluation pipeline 互通。

8.6.3 Verifier-Based and Training-Time Evaluation

R2E-Gym、verifiers 等系统把环境反馈作为训练或优化信号。Meta-Harness 更进一步，把 harness design 本身作为搜索对象：不是只问哪个模型最好，而是问什么 prompt、tool interface、control loop 更可靠。

8.7 Summary

V 层把 agent 行为变成可用工程证据。最终成功率仍有用，但不够。长程 agent 评测要回答：为什么成功或失败，过程是否可接受，evaluator 是否可信，下一个应该改哪个 harness 组件。

9. Governance and Security (G)

G 层处理 agent 行为如何被约束、保护和追责。agent 现在能执行命令、发邮件、提交代码、调用 API，因此必须回答：它能访问什么，谁授权，什么必须禁止，什么时候需要人类确认，出事后如何审计。

9.1 Permission Models and Identity Management

第一类是静态权限边界。Codex、Gemini CLI 等 coding agent 通常限制工作区文件访问、网络访问、命令 allow/deny list。这种方式清晰易查，但无法表达任务特定意图。

第二类是上下文相关 privilege control。Progent 用 DSL 表达工具名、参数、环境状态上的 predicate，在每次调用前判断是否允许。Conseca 从可信上下文生成 task-specific policy，再用独立 deterministic checker 执行。这种“生成策略”和“执行策略”分离很重要，因为执行器应可审计。

第三类是身份管理和 inter-agent access control。多 agent 系统需要知道谁在请求访问。相关工作提出 User ID Token、Agent ID Token、Delegation Token，把用户意图、agent 身份和权限范围绑定起来。SAGA 用短期访问 token 和 cryptographic keys 管理多 agent 接触策略。IsolateGPT 用 hub-and-spoke 隔离第三方应用。

凭证管理也是核心。API key、session token、OTP 不应进入 LLM 上下文。常见模式是 secret vault 存真实凭证，LLM 只看到 placeholder，由自动化层执行时替换。

Web-level permission coordination 则试图跨组织边界管理 agent 行为。agent-permissions.json 类似 robots.txt，让网站声明哪些 UI 元素允许 agent 操作、速率限制、哪些动作要人类确认。

9.2 Lifecycle Hooks

权限模型定义什么允许，hook 定义何时检查。

H1 输入进入 LLM 前检查。PromptShield、DataSentinel 检测 prompt injection。规则检测严格但脆弱；模型检测泛化强但慢，也可能被自适应攻击绕过。

H2 工具调用前检查。ShieldAgent 将安全约束形式化为可验证 predicate；ControlValve 在多 agent 中约束控制流，防止一个 agent 非法重定向到另一个 agent。

H3 工具输出回到上下文前检查。CaMeL 用 capability-based information flow control，给每个数据值标注 provenance，区分可信用户输入和不可信网页内容，防止不可信数据影响控制流。

H4 人类确认。Codex、Gemini CLI、Cursor、OpenHands 等对破坏性或越界动作要求用户批准。这里有 UI 风险：如果提示太频繁，用户会习惯性批准；如果太少，覆盖不够。

开放问题是 hook API 不统一，多个 hook 堆叠后可能干扰。上游 sanitizer 可能改变下游 detector 需要的信号，使组合比单独使用更弱。

9.3 Component Hardening

组件加固包括模型边界和工具边界。

模型加固方面，instruction hierarchy 训练模型区分系统指令、用户消息、工具输出等不同优先级。SecAlign 用偏好优化让模型在 prompt injection 下拒绝错误指令。Llama Guard 等外部分类器可在运行时筛查输入输出，优势是分类 taxonomy 可独立更新，能跨多个 LLM 后端复用。

工具加固方面，MCP 是重点。攻击研究显示恶意 MCP tool description 可能在用户调用工具前就影响模型行为，或者诱导工具执行恶意代码、窃取凭证。ETDI 通过签名和版本化工具定义，防止 rug-pull：一个先前批准的工具有被悄悄更新成恶意版本。

供应链风险不容忽视。LLM 生成代码时可能幻觉不存在的包名，攻击者注册这些包名后植入恶意代码，即 slopsquatting。工具签名只能覆盖 MCP tool，本地包管理器、检索源、数据集仍需要治理。

9.4 Declarative Constitutions

把治理逻辑硬编码在应用代码中不利于审计和修改。声明式 constitution 把规则外部化到 YAML 或 DSL。

训练时 constitution，如 Anthropic Constitutional AI，在模型对齐阶段影响默认行为。它将原则分层，例如安全、伦理、合规、帮助性。部署后修改成本高，也难以由 harness 独立审计。

部署时 constitution，如 AutoHarness YAML，指定 pipeline mode、risk pattern、allowed/denied tool、token budget、audit destination。优点是可读、可 diff、可版本化，安全和合规团队可以审查。

更复杂策略需要 DSL 或形式化规范。Progent policy language 支持布尔 predicate、量词和环境引用。Formal-LLM 用 pushdown automata 表达顺序约束。VeriSafeAgent 用 UI 状态转移 DSL 验证 GUI 动作是否符合用户意图。

开放问题：没有通用 schema；constitution 内部一致性和完整性缺少验证工具；训练时 alignment 和部署时 rule 冲突时如何处理仍不清楚。

9.5 Audit Infrastructure

审计基础设施记录 agent 做了什么、为什么、策略如何决策、是否合规。理想 audit record 包括 trace id、principal identity、tool invocation、policy decision/version、execution result、resource cost、输入输出 integrity hash。

检测分 per-action 和 trajectory-level。Per-action detector 每个工具调用单独判断，便宜且可内联，但发现不了多步慢速攻击。Trajectory-level detector 看动作序列，能发现分布式攻击和异常交互模式，但延迟高、定位难。

成本和资源审计也属于治理。Runaway loop 或 multi-agent fan-out 可能快速耗尽预算。AutoHarness 等系统通过 constitution 设置 session-level budget，并记录 token/latency。

Tiered governance pipeline 是一种趋势：低风险场景只做 parse-risk-permission-execute-audit，高风险场景再加 context enrichment、output validation、anomaly scoring、human escalation、formal constraint verification。

9.6 Situating Governance in the Agent Security Landscape

作者将治理机制放入更大 agent security landscape。风险包括 untrusted interfaces、wrong instruction following、unconstrained data flow、hallucination、data leakage、unauthorized action、resource drain。

权限模型缓解数据泄露和未授权动作；输入护栏缓解不可信接口和错误指令服从；输出护栏缓解幻觉、泄露和未授权动作；信息流控制缓解无约束数据流；监控审计发现数据泄露、未授权动作和资源耗尽；human-in-the-loop 处理关键决策。

现实系统覆盖不完整。很多系统记录工具调用，但缺少自动异常检测；信息流控制、身份管理、形式化验证更少见。作者认为 agent security 需要 defense-in-depth，但层叠防御必须可组合，否则会互相干扰。

9.7 Research Directions

G 层开放方向有八个：

- 标准化 policy 和 audit language，使治理模块可移植。
- 形式化治理保证，例如证明 constitution 无矛盾、覆盖所有工具类型。
- 自适应治理，根据任务上下文生成策略，但策略生成器本身也要被治理。
- 长程 agent 治理，支持权限续期、撤销、会话级审计。
- 可用治理界面，包括权限 UI、审计 dashboard、constitution editor。

- 跨层治理一致性，协调训练时 constitution、部署时 YAML、运行时 hook。
- 端到端供应链治理，覆盖 package、dataset、retrieval source、tool。
- 统一 adversarial benchmark，报告防御效果、误报率和开销。

10. Cross-Cutting Concerns

第 10 章强调七层不是独立模块。生产失败常发生在层与层之间。

执行环境会限制生命周期编排。例如 full desktop VM 启动慢，会影响多 agent 并行策略；轻量 OS permission sandbox 适合本地开发，但不适合恶意代码评测。

上下文管理会影响评测复现。如果 memory 未 reset，benchmark 可能发生状态泄露；如果 compaction policy 改变，同一模型得分也会变。

治理横跨所有层。权限和身份必须跟工具调用、沙箱动作、trace、audit、human approval 一起记录，否则事后无法问责。

标准化协议带来互操作，也把责任转给 O/G 层：标准工具调用只有在 provenance、permission、cost、failure evidence 能跨系统保留时才真正有用。

作者列出五个反复出现的生态缺口：cross-tool interoperability、cost attribution、failure recovery、multi-repository orchestration、human-agent handoff。

11. Cross-Layer Synthesis

11.1 Cost-Quality-Speed Trilemma

可靠 harness 受成本、质量、速度三角约束。更强沙箱提高安全和复现，但启动慢、成本高；更丰富上下文和记忆改善连续性，但增加 token 和检索延迟；更深评测和观测改善诊断，但增加存储、标注和处理成本。

生产系统不能把 quality 当唯一目标，必须决定哪些检查同步执行，哪些离线跑，哪些高风险任务才启用昂贵控制。

11.2 Capability-Control Tradeoff

agent 能力越强，控制问题越难。更多工具扩大覆盖，也扩大选择错误和攻击面；持久记忆支持长任务，也带来 provenance、staleness、privacy 风险；开放沙箱提升自主性，也扩大 blast radius。

这不是安全附属问题，而是贯穿工具 schema、context policy、runtime permission、identity、auditability、human approval 的主设计轴。

11.3 Harness Coupling Problem

harness 各层强耦合。执行环境改变 package availability、reset semantics、latency 和 failure mode，从而改变评测结果。工具描述消耗上下文并影响模型行为。Trace 只有记录身份和权限时才能成为治理证据。评测设计会奖励某些 recovery loop，惩罚另一些。

因此，prompt、tool、memory、sandbox、verifier、monitor 的改动都应作为系统变更测试，而不是只看局部指标。

11.4 From Agent Frameworks to Agent Platforms

生态正从 framework 走向 platform。Framework 提供本地抽象：agent、tool、memory、loop。Platform 提供持久 workspace、托管沙箱、身份、计费、可观测、评测、治理、人类交接、多租户和合规。

这意味着问题从“如何写一个 agent”变成“如何运营一组 agent，使它们的行动长期可检查、可恢复、可逆、可追责”。

11.5 Open Research Agenda

作者提出 harness-as-adaptive-control-system 的研究议程。需要：

- benchmark 不只换模型，也系统改变 harness intervention。
- trace-native failure attribution。
- agent、tool、sandbox、evaluator、human 之间的 handoff protocol。
- 随模型升级自动简化 harness 的优化方法。

12. Open Problems and Future Directions

12.1 Hardening and Scaling Execution Environments

执行环境是安全、规模和可移植性相遇的边界。前沿模型已经能在部分配置中利用 sandbox 弱点，但防御研究仍分散。大规模训练和评测中，一任务一容器模式成本很高；learned surrogate environment 可能降低成本，但保真度仍未知。

未来需要统一安全评测、运行时成本模型和可移植层。系统应根据任务和威胁模型选择 container、microVM、OS-level permission、desktop VM、browser environment 或 learned surrogate。

12.2 Maintaining Reliable State in Long-Running Agents

长程 agent 的核心不是塞更多 token，而是让 agent 工作状态与真实任务状态保持一致。每次 summary、retrieval、compaction、forgetting 都可能丢约束、扭曲优先级或保留过期假设。

作者建议把 context management 重新看作 state estimation。未来需要 uncertainty-aware summary、memory provenance、contradiction handling、staleness marker、从持久 artifact 重建缺失状态的 recovery procedure。

12.3 Diagnosing Failures from Agent Traces

最终分数过于粗糙。失败可能来自模型、工具、沙箱、上下文、benchmark、judge、orchestration。评测层应成为 measurement instrument，而不只是 leaderboard。

Trace-native evaluation 应让 trace 成为 outcome score、trajectory quality、failure attribution、regression test 的主对象。生产异常 trace 应自动变成可复现测试，用于改 prompt、tool、context 和 orchestration。

12.4 Standard Handoffs Across Agents, Tools, and Humans

现代 harness 将任务分给 planner、subagent、tool、sandbox、evaluator、human，但交接接口仍很随意。仅传文本 summary 不够。标准 handoff 应包含 intent、constraints、permissions、artifacts、provenance、budget state、risk level、trace history、unresolved decisions。

这个问题既技术也组织。协议要足够丰富以支持安全和恢复，又要足够简单以普及。它还必须明确责任：谁授权，接收者能做什么，何时必须交回人类或上级 agent。

12.5 Keeping Harnesses Useful as Models Improve

harness 不应单调变复杂。每个 wrapper、reset、verifier、planner、memory rule、permission gate 都隐含模型能力不足的假设。模型升级后，某些控制可能变成不必要成本。

未来 harness 需要自我优化和自我简化：通过消融、A/B test、shadow mode 判断哪些 intervention 仍对质量、安全、可靠性有因果贡献。风险是 benchmark overfitting，因此目标应是 adaptive simplification，而不是只针对少数 benchmark 堆 scaffold。

13. Conclusion

论文结论是：agent harness 是独立工程表面，现实可靠性的上限不只由模型能力决定。ETCLOVG 将生产团队已经在实践中分化出的职责形式化。公开项目映射说明，执行、工具、生命周期和评测生态较成熟，可观测和治理仍较薄弱。

从 prompt engineering 到 context engineering 再到 harness engineering 的演进，说明 agent 工程的对象正在从“单次模型输入”变成“长程闭环控制系统”。作者也承认语料偏向英文、开源、GitHub 可见和 coding agent；ETCLOVG 目前更像描述框架，未来应发展成能指导设计决策的规范框架。

图表详解

Figure 1: Prompt、Context、Harness Engineering 对比

这张图说明工程关注点的扩大。Prompt engineering 优化单次输入；context engineering 管理每一步的信息状态；harness engineering 管理完整闭环，包括执行、工具、状态、观测、评测和治理。汇报时可以用它作为开场图，解释为什么本文不是 prompt 技巧综述。

Figure 2: ETCLOVG 总体架构示意

图中 E/T/C/L 是结构支柱，O/V/G 是跨系统控制层。O 提供监控，V 提供评测和反馈，G 对整个系统施加安全和治理约束。重点是 O/G 被画成独立层，而不是挂在 L 下的 hook。

Figure 3: 2022-2026 时间线

时间线展示从早期单 loop agent 到完整 harness 基础设施的演进。早期重点是 ReAct 和自主循环，中期是工具、多 agent、benchmark，后期是沙箱、可观测、治理、长生命周期和 platform。

Figure 4: ETCLOVG 分类树

这张图是全文目录式地图。每个分支对应后续章节：E 有七类沙箱；T 有协议、工具发现、工具训练、session；C 有短/中/长期记忆和 drift；L 有单 agent、多 agent、全生命周期；O 有 trace、ops、cost、reliability；V 有五阶段 eval；G 有权限、hook、加固、constitution、audit。

Figure 5: 语料构建流程

图示项目从 GitHub、论文、curated list、包注册表和公司工程资料进入候选池，经过去重、纳入/排除标准检查，再映射到 ETCLOVG 层。它支撑论文经验贡献。

Figure 6: E 层代表工作

按沙箱类别列出 Daytona、E2B、Modal、Anthropic Computer Use、Judge0、OpenAI Code Interpreter、OpenHands、WebArena、Claude Code sandboxing、IsolateGPT、SWE-ReX、SandboxEscapeBench 等。读图时重点看类别差异，而不是记项目名。

Figure 7: T 层代表工作

展示 MCP、A2A、function calling、OpenAPI、工具检索、工具增强训练和 session 管理。它说明工具层不仅是 API 调用，还包括工具选择、协议边界和模型侧工具能力。

Figure 8: C 层代表工作

按时间尺度组织：短期 active context、中期 session state、长期 persistent memory、long-horizon context techniques、context drift。它对应本章从“当前窗口”到“长期记忆”的叙述。

Figure 9: L 层代表工作

分为 single-agent inner loop、multi-agent orchestration、full lifecycle pipeline。Table 2 则进一步给出系统、GitHub stars、主要模式和执行模型。

Figure 10: O 层代表工作

包括 tracing/monitoring、agent-specific ops、cost tracking、reliability engineering、unified observability。重点是从“记录日志”升级到“诊断、成本归因和自动修复”。

Figure 11 和 Figure 12: V 层五阶段生命周期

Figure 11 列代表系统，Figure 12 画质量控制循环。核心是 eval 不停在最终分数，而是从任务 grounding 到 readiness、trace、judgement、regression feedback。

Figure 13 和 Figure 14: G 层与治理 hook

Figure 13 列 governance/security 代表工作。Figure 14 画一个 tool-use cycle 中四个 hook：输入前、动作前、工具输出后、人工确认。它是理解 G 层最直观的图。

表格详解

Table 1: 工具/接口标准按边界分类

这张表的重要性在于澄清 MCP、A2A、function calling、OpenAPI、AGENTS.md 等不是同类替代品。它们跨越不同边界：模型到函数、agent 到能力、agent 到 agent、agent 到仓库/环境。比较协议时必须先问“它解决哪个边界问题”。

Table 2: 编排系统比较

表中把 OpenCode、Claude Code、Gemini CLI、Codex CLI、Aider、SWE-agent 归为单 agent loop；把 AutoGen、LangGraph、Semantic Kernel、OpenAI Agents SDK、DeepAgents 等归为多 agent 编排；把 Vibe Kanban、Symphony、GitHub Agentic Workflows 归为全生命周期 pipeline。执行模型包括 stateless replay、stateful、hybrid。

Table 3: 治理机制与风险映射

表格把 permission、guardrail、information flow control、component hardening、monitoring/audit、human-in-the-loop、privilege separation、formal verification 映射到不同风险。它说明没有单一治理机制能覆盖所有风险，必须 defense-in-depth。

Table 4: 治理功能覆盖

表格展示现实系统在 permission、hooks、hardening、constitution、audit、multi-agent governance 上覆盖很不均衡。结论是治理仍是安全部署瓶颈之一。

可直接用于汇报的总结

这篇论文可以用一句话概括：LLM agent 的可靠性不只是模型问题，而是模型外层 harness 的系统工程问题。

如果要给导师讲 3 分钟，可以这样讲：

- 背景：早期 agent 研究把重点放在模型能力，但生产实践发现，长程任务能不能完成很大程度取决于执行底座。
- 核心观点：harness 是 binding constraint。固定模型、只改工具、上下文、验证 hook、沙箱和编排，也能显著改变 benchmark 分数。
- 方法：作者提出 ETCLOVG 七层框架，将 agent harness 拆成执行、工具、上下文、生命周期、可观测、验证、治理。
- 发现：执行、工具、编排、评测生态最成熟；可观测和治理在开源中较薄；真正难点在跨层耦合。
- 意义：未来 agent 工程会从写 prompt、调工具，转向运营可观测、可验证、可治理的长程控制系统。

如果要讲创新点，可以说：

- 它把 harness engineering 从实践术语整理成研究对象。
- 它把 Observability 和 Governance 提升为独立层。
- 它用公开项目映射说明生态覆盖和缺口。
- 它提出 cost-quality-speed、capability-control、harness coupling 三个跨层问题。

如果要讲不足，可以说：

- 语料偏向开源、英文、GitHub 和 coding agent。
- 分类框架偏描述性，还不能直接告诉工程师如何选择方案。
- 很多代表项目和数据是 2026 年左右快速变化的生态快照，需要持续更新。
- 对不同层之间的因果关系还缺少严格实验验证。

术语速查

- Agent harness：把 LLM 包装成可执行、可恢复、可观测、可治理 agent 的系统底座。
- Binding constraint：当前限制系统可靠性的主要瓶颈。
- Execution environment：agent 动作发生的运行环境。
- Sandbox：限制 agent 动作影响范围的隔离或权限边界。
- Tool protocol：工具描述、发现和调用的通信规范。
- Context engineering：管理每一步模型可见信息的工程。
- Memory system：跨轮次或跨会话保存和检索信息的系统。
- Lifecycle state：harness 自身推进任务所需的操作状态。
- Orchestration：单 agent、多 agent 或全生命周期任务的控制流组织。
- Observability：记录和分析系统行为、成本、失败和性能的能力。
- Trace-native evaluation：以完整执行轨迹为评测和诊断对象。
- Governance：权限、身份、策略、审计、人类确认和安全控制。
- Context rot：单次长上下文输入中的性能退化。
- Context drift：长程执行中状态和目标逐渐偏移。
- Handoff：agent、工具、子系统和人之间的责任与状态转移。
- Capability-control tradeoff：能力扩展与控制难度之间的权衡。
- Harness coupling：harness 各层相互影响导致局部优化不一定全局有效。

附录 A：逐层深读笔记

这一部分不是新增论文结论，而是把前文内容再拆细，帮助第一次读这类 agent harness 综述的人建立“看系统”的方式。你可以把它当成阅读原文时的旁注。

A.1 E 层：为什么执行环境不是普通 DevOps 问题

传统软件系统中，执行环境常被视为部署或运维问题；但在 agent 系统里，它直接影响模型行为本身。因为 agent 的每一步动作都要通过环境反馈来决定下一步。如果环境不可复现，模型得到的观察就不可复现；如果沙箱太弱，错误动作会产生真实破坏；如果环境太慢，agent 的长程规划会被超时和成本限制打断。

理解 E 层时，可以抓住三个问题：

- 这个 agent 到底能触碰哪些资源：文件、网络、浏览器、桌面、数据库、凭证、外部 API。
- 环境能否被重置：benchmark、训练、回归测试都依赖 reset。

- 隔离边界是否匹配威胁模型：prompt injection、恶意代码、误操作、资源耗尽对应的防御不同。

通用托管沙箱适合多租户、弹性任务和需要大量短生命周期环境的场景。它们的价值不是“能跑命令”这么简单，而是把执行能力变成 API。这样 agent harness 可以动态创建、销毁、记录和替换环境。

Computer-use 沙箱适合没有 API 的真实软件世界。它的难点是动作空间巨大：点击哪里、看什么、如何判断界面状态、如何处理弹窗和视觉误识别。它也让安全问题更复杂，因为 GUI 应用经常与真实账号、文件、浏览器 session 绑定。

代码沙箱适合高吞吐评测。比如 coding benchmark 通常要并行跑成千上万个任务，要求启动快、清理快、资源隔离清楚。它与通用沙箱的差异是牺牲任务泛化，换取代码执行吞吐和验证效率。

OS 级权限沙箱适合本地开发型 agent。比如一个 coding assistant 在你的仓库里工作，不一定需要完整 VM，但需要防止它访问家目录、SSH key 或乱连外网。这类沙箱更像“权限收窄器”，而不是“全新环境”。

沙箱抽象层代表生态成熟。早期系统会把 Docker 或某个 vendor API 写死；成熟 harness 会把执行后端抽象成可替换接口。这样当任务从本地开发变成云端评测，或从低风险任务变成高风险任务时，系统可以更换执行 substrate。

A.2 T 层：工具不是越多越好

工具层最容易被误解为“给 agent 接更多 API”。论文实际强调的是工具接口质量和动作空间控制。

工具定义进入上下文后，会成为模型决策的一部分。工具描述写得模糊，模型会误用；工具数量过多，模型会选择困难；工具参数设计不符合自然语言推理习惯，模型会填错；工具返回太长，会污染后续上下文；工具错误信息不清楚，模型无法恢复。

一个好的 agent-facing tool 通常有这些特点：

- 名称直接表达意图，而不是内部系统名。
- 描述说明什么时候用、什么时候不用。
- 参数少而语义清楚，避免让模型猜复杂嵌套结构。
- 返回值包含足够证据，但不把无关日志塞满上下文。
- 错误信息可行动，告诉 agent 下一步如何修正。
- 支持幂等或可恢复，避免重试造成状态不一致。

MCP 的意义在于把工具生态标准化，但标准化不自动等于安全和可靠。一个 MCP server 可以暴露工具，也可能暴露恶意描述、过宽权限或不稳定状态。因此 T 层必须与 G 层和 O 层结合：工具来源要可信，调用要被授权，结果要被记录。

工具检索和动态选择是 agent 扩展的关键。如果企业有上千个内部 API，不可能每次都暴露给模型。更合理做法是先根据任务、权限、上下文检索候选工具，再把小集合暴露给模型。这类似信息检索问题，但目标不是找文档，而是找可行动能力。

工具增强训练解决的是模型侧能力，工具协议解决的是系统侧接口。两者不能互相替代。一个模型即使用过很多 API 训练，如果运行时 schema 糟糕、权限错误、工具状态不一致，也会失败；反过来，工具协议再好，模

型不会识别何时调用，也不能可靠完成任务。

A.3 C 层：上下文管理是“信息预算管理”

上下文窗口看似只是 token 限制，实际是 agent 的工作记忆和注意力预算。每塞入一段内容，都会挤占其他内容，并改变模型注意力分布。

短期上下文管理像“当前工作台整理”。你要把当前任务需要的文件片段、错误日志、工具说明、约束和下一步目标放在最容易被模型使用的位置。无关历史、重复日志、过长工具输出会让模型迷失。

中期状态像“项目笔记”。当任务跨越上下文清理或多次运行时，agent 需要一份外部化的进度记录。它不需要像数据库那样复杂，但必须准确、简洁、可恢复。常见文件包括 progress.md、todo.md、decision-log.md、known-issues.md。

长期记忆像“经验库”。它适合保存用户偏好、过去任务经验、常见错误、组织知识、跨项目事实。长期记忆的难点不是存储，而是写入质量、检索质量、遗忘策略、矛盾处理和 provenance。

Context drift 是长程 agent 最危险的问题之一。一个 agent 可能没有明显报错，但逐渐忘记最初约束、重复做过的事、基于过期假设继续行动。这个问题很像导航偏航：每一步误差很小，但 100 步后已经离目标很远。

因此，好的 C 层通常要和 V/O/G 层结合：

- O 层记录上下文变化和压缩结果。
- V 层检查压缩后是否仍能完成任务。
- G 层防止不可信内容进入关键控制路径。
- L 层在 checkpoint 时固化真实任务状态。

A.4 L 层：编排不是“多套几个 agent”

Lifecycle and Orchestration 关注的是任务如何从开始走到结束。它不是简单地把多个 agent 串起来，而是定义任务状态、控制流、恢复策略和责任边界。

单 agent loop 的优点是简单、可检查、开销低。很多实际 coding agent 都从这里开始。它的缺点是所有职责集中在一个上下文里：理解需求、探索仓库、修改代码、运行测试、解释错误、判断完成。这会让上下文膨胀，也容易让 agent 自我确认过度。

多 agent 编排的价值是分工。planner 负责拆任务，executor 负责执行，critic/evaluator 负责检查，summarizer 负责压缩，researcher 负责探索。分工可以提升复杂任务的结构性，但同时带来通信成本和状态一致性问题。

全生命周期 pipeline 更接近真实工程流程。一个任务不是“模型输出代码”就结束，而是要进入 issue、branch、commit、test、review、PR、merge、rollback 等流程。这里 harness 需要与 Git、CI、ticket system、review tool、artifact store 集成。

判断一个 L 层设计是否成熟，可以问：

- 它是否有明确任务状态机。
- 它是否知道何时停止。

- 它能否从失败处恢复，而不是从头开始。
- 它能否把未完成状态交给另一个 agent 或人类。
- 它是否保留足够证据供评测和审计。

A.5 O 层：没有 trace，就没有调试

可观测性在 agent 系统里比传统应用更重要，因为 agent 行为具有非确定性、长轨迹和语义决策。传统日志能告诉你某个 API 返回 500，但 agent trace 还要告诉你为什么 agent 选择了这个 API、当时看到什么上下文、工具返回后如何影响下一步。

一个有用的 agent trace 至少应包含：

- 任务输入和初始环境版本。
- 每次模型调用的模型、参数、token、成本、延迟。
- 上下文构造摘要，包括系统提示、工具列表、检索内容、压缩状态。
- 工具调用名称、参数、返回、错误和重试。
- 环境状态变化，如文件 diff、浏览器状态、测试结果。
- 权限决策和人工确认记录。
- 最终输出、评测结果和失败归因。

成本观测也不只是财务问题。成本突然升高可能表示 agent 陷入循环、工具返回过长、上下文缓存失效、路由到过强模型、检索策略过宽。成本和可靠性经常是同一个问题的两个信号。

论文提到 unified observability 的方向很重要：trace、eval、prompt version、tool version、sandbox version、policy version 应该连接起来。否则你知道某次失败发生了，却不知道是哪次配置变更导致的。

A.6 V 层：评测是质量控制循环，不是排行榜

很多 agent 论文用 benchmark 分数说话，但作者提醒：agent 分数不是模型单独属性，而是 model-harness pair 属性。同一模型换工具、换沙箱、换上下文策略、换 retry 机制，得分都会变。

Task grounding 是评测可信的第一步。一个任务必须明确：

- 初始环境是什么。
- agent 能用哪些工具。
- 哪些动作允许或禁止。
- 成功条件是什么。
- 失败如何判定。
- 是否允许人工介入。
- 成本和时间限制是什么。

Readiness validation 是很多 leaderboard 忽视的环节。如果测试依赖坏了、沙箱没 reset、网页服务不可用、memory 泄露、权限配置变化，那么失败不应算模型失败。评测 harness 自己也需要被测试。

Trace-native evaluation 的价值是把“失败了”变成“为什么失败”。例如同样是 SWE-bench 未通过，可能原因包括没找到相关文件、改错文件、测试没跑、依赖没装、patch 语义错、benchmark 测试 flaky、agent 过早停止。没有 trace，就只能猜。

Regression feedback 是生产系统最需要的部分。每次 prompt、tool schema、sandbox image、policy、judge prompt 改动都可能引发回归。成熟 harness 应该像软件工程一样，有单元测试、集成测试、回归测试和生产监控。

A.7 G 层：治理是让 agent 可部署的前提

治理层回答“agent 被允许做什么”。这不是给系统加几个安全提示词，而是完整的权限、身份、策略、审计和人类责任体系。

权限模型要处理三个层次。第一是静态边界，如工作区目录和网络 allowlist。第二是上下文策略，如某个任务允许访问某个 API，但另一个任务不允许。第三是跨 agent 和跨组织委托，如一个 agent 能否代表用户请求另一个服务。

Lifecycle hook 是治理插入点。输入进入模型前检查，可以防 prompt injection；工具调用前检查，可以防危险动作；工具输出进入上下文前检查，可以防不可信数据污染；高风险动作前人工确认，可以保留人类责任。

组件加固降低 hook 压力。模型越能抵抗 prompt injection，工具越有签名和版本控制，供应链越可信，治理 pipeline 需要拦截的异常越少。但任何单点加固都不够，因为模型、工具、数据、包、浏览器、用户界面都可能成为攻击面。

审计是治理闭环。没有审计，就无法回答事故后问题：谁发起任务，agent 为什么调用这个工具，哪个策略允许了动作，返回了什么，是否有人批准，成本是多少，哪些数据被访问。

附录 B：原文代表系统按用途重排

这一节按“你要解决什么问题”重排论文中的代表系统，方便学习时建立工具地图。

B.1 如果你关心 agent 在哪里执行

通用云沙箱：Daytona、E2B、Modal、Northflank、OpenSandbox、Docker Sandboxes。它们适合把 shell/code execution 作为可编程资源提供给 agent。

本地/OS 权限沙箱：Anthropic sandbox-runtime、Claude Code sandboxing、IsolateGPT。它们适合本地开发和权限收窄。

完整桌面或 computer-use：Anthropic Computer Use、CUA、OSWorld。它们适合 GUI 操作和无 API 应用。

浏览器环境：WebArena、VisualWebArena、BrowserGym、WorkArena。它们适合 web agent 评测，也适合研究网页 prompt injection。

沙箱抽象：SWE-ReX、smolagents executors、Kubernetes Agent Sandbox。它们解决后端可替换和跨环境执行问题。

B.2 如果你关心工具调用和协议

接口标准：Function calling、OpenAPI、MCP、A2A、ACP/ANP、AGENTS.md/AGENT.md。

工具发现和选择：EASYTOOL、AnyTool、CRAFT、MetaTool、MCP-Zero、ToolRet、ToolRegistry、SkillRouter、SkillRet。

工具能力训练：Toolformer、Gorilla、ToolLLM/ToolBench、ToolkenGPT、CREATOR。

工具运行框架：LangChain、Semantic Kernel、smolagents、OpenAI Agents SDK。

B.3 如果你关心上下文和记忆

短期上下文实践：Anthropic Effective Context Engineering、Manus Context Engineering、Claude Code context management、context-space。

中期跨运行状态：structured note-taking、planning-with-files、Trellis、claude-mem、everything-claude-code。

长期记忆系统：MemGPT、Generative Agents、MemoryBank、Mem0、A-MEM、Hindsight、Honcho、Mozilla cq。

长程上下文技术：context compaction、sub-agent context isolation、checkpoint/resume、tool result clearing。

context drift 评测：MemBench、MemoryArena、incremental memory evaluation。

B.4 如果你关心编排

单 agent coding loop：OpenCode、Claude Code、Gemini CLI、Codex CLI、Aider、SWE-agent。

多 agent 编排：AutoGen、LangGraph、Semantic Kernel、OpenAI Agents SDK、DeepAgents、DeerFlow、Hive、Emdash。

全生命周期任务流：Vibe Kanban、Symphony、GitHub Agentic Workflows。

B.5 如果你关心可观测和运维

Trace 平台：Langfuse、Opik、Arize Phoenix、MLflow。

Instrumentation 标准：OpenTelemetry、OpenLLMetry、OpenInference。

系统级观测：AgentSight、AgentTrace。

AgentOps 平台：AgentOps、RagaAI Catalyst、Laminar。

成本优化：TensorZero、Helicone、FrugalGPT、GPTCache、QC-Opt、Dual-Pool Token-Budget Routing。

可靠性与故障分析：MAST、AgentErrorTaxonomy、AgentDebug、SentinelAgent、AgentFixer、QSAF。

B.6 如果你关心评测

软件工程：SWE-bench、Terminal-Bench、SWE-agent、SWE-ReX、Repo2Run。

浏览器和桌面：WebArena、VisualWebArena、BrowserGym、WorkArena、OSWorld。

跨域任务：AgentBench、GAIA、The Agent Company、WorkArena++。

评测框架：promptfoo、DeepEval、RAGAS、lm-evaluation-harness。

训练/优化型评测：R2E-Gym、verifiers、Meta-Harness。

B.7 如果你关心治理和安全

权限与身份：Progent、Conseca、SAGA、IsolateGPT、agent-permissions.json。

输入/输出护栏：PromptShield、DataSentinel、ShieldAgent、ControlValve。

信息流控制：CaMeL、SAFEFLOW。

模型和分类器加固：instruction hierarchy、SecAlign、Llama Guard。

MCP 和工具安全：McpSafetyScanner、Trail of Bits MCP security analysis、ETDI。

声明式策略：Anthropic Constitutional AI、AutoHarness、AgentSpec、Formal-LLM、VeriSafeAgent。

审计和异常检测：AutoHarness audit、AgentMonitor、AgentAuditor、SentinelAgent、OWASP LLM Top 10。

附录 C：每章学习问题

这些问题可以用来检查自己是否真的读懂了综述。

C.1 Introduction

- 为什么作者认为 agent 可靠性不能只归因于模型？
- harness-only gain 为什么能支持 binding-constraint thesis？
- practitioner-research gap 指的是什么？
- 为什么 harness engineering 可以看作 prompt engineering 和 context engineering 的上层扩展？

C.2 Background and Taxonomy

- ReAct 时代暴露了哪些基础设施问题？
- 工具集成、多 agent、benchmark 标准化分别推动了什么？
- ETCLOVG 七层中，哪些是结构核心，哪些是控制平面？
- 为什么 Observability 和 Governance 需要独立出来？
- 论文语料有哪些偏差？这些偏差会如何影响结论？

C.3 Execution Environment

- agent 沙箱的安全、复现、活性三重作用分别是什么？
- OS 级权限沙箱和 microVM 沙箱适合的威胁模型有什么不同？
- 为什么 browser benchmark 同时属于执行层和评测层？

- 沙箱抽象层为什么代表生态成熟？
- sandbox escape 对 agent 研究意味着什么？

C.4 Tool Interface

- MCP 和 A2A 解决的是同一个问题吗？
- 为什么工具菜单过大会降低可靠性？
- tool retrieval 与 document retrieval 有什么相似和差异？
- 为什么工具返回 evidence-bearing artifact 比返回 yes/no 更好？
- sessionful tools 会引入哪些错误？

C.5 Context and Memory

- context rot 和 context drift 有何区别？
- 为什么更大上下文窗口不能自动解决记忆问题？
- 短期、中期、长期 memory 分别解决什么问题？
- KV-cache 友好设计为什么影响生产成本？
- 为什么 context management 需要和 verification/governance 结合？

C.6 Lifecycle and Orchestration

- lifecycle state 与 model context 有什么区别？
- stateless replay 和 stateful execution 各有什么优缺点？
- 多 agent 编排的收益和成本分别是什么？
- issue-to-PR pipeline 比单次代码生成多了哪些工程约束？
- 如何判断一个 agent run 应该停止？

C.7 Observability

- agent trace 应该记录哪些信息？
- 为什么成本异常可能是可靠性问题信号？
- cognitive observability 想解决什么问题？
- observability 和 evaluation 脱节会造成什么后果？
- harness-as-assumption principle 对模型升级有什么启示？

C.8 Verification and Evaluation

- 为什么 agent 分数是 model-harness pair 的属性？
- task grounding 不充分会导致什么评测问题？
- readiness validation 为什么重要？
- outcome-level、trajectory-level、evaluator-level evaluation 分别解决什么？
- 如何把生产失败 trace 变成 regression test？

C.9 Governance and Security

- 静态权限和上下文相关权限有什么差别？
- 四个 hook 点分别拦截什么风险？
- 为什么信息流控制比普通输入过滤更强？
- deployment-time constitution 相比 training-time constitution 有何优势？
- 审计记录最少应该包含哪些字段？

C.10 Cross-Layer Synthesis

- cost-quality-speed trilemma 在各层分别如何体现？
- capability-control tradeoff 为什么不是单纯安全问题？
- harness coupling 会怎样让局部优化变成全局退化？
- agent framework 和 agent platform 的差别是什么？
- 为什么未来 harness 需要 adaptive simplification？

附录 D：适合写进读书报告的段落

下面这些段落是中文研读后的表达模板，可以直接改写进读书报告或组会讲稿。

D.1 研究背景模板

近年来，LLM agent 从简单的 prompt-driven demo 逐渐走向能够执行代码、调用工具、操作浏览器和完成长程任务的系统。随着任务复杂度提升，可靠性瓶颈不再只来自模型是否足够强，而越来越多地来自模型外层的执行基础设施。该论文将这层基础设施称为 agent harness，并将其视为连接模型能力与真实任务执行之间的关键工程层。

D.2 核心贡献模板

论文的核心贡献是提出了一个面向 LLM agent 系统的 harness engineering 视角。作者首先提出 binding-constraint thesis，认为长程 agent 可靠性常受 harness 而非模型单体约束；其次提出 ETCLOVG 七层分类，将执行环境、工具接口、上下文、生命周期、可观测、验证和治理统一在一个框架中；最后通过大量公开项目映射，展示当前 agent harness 生态的覆盖密度和薄弱环节。

D.3 方法论评价模板

这篇综述的价值在于它不是按模型能力分类，而是按系统职责分类。与传统 agent 综述关注 planning、memory、tool use 等模型侧能力不同，本文把沙箱、协议、trace、评测、权限和审计纳入同一工程框架，使 agent 系统更接近可部署软件系统而不是孤立模型实验。

D.4 局限性模板

论文也存在局限。首先，项目语料偏向英文、开源和 GitHub 可见项目，可能低估闭源生产系统的成熟实践。其次，ETCLOVG 目前主要是描述性分类，尚未发展成可以直接指导工程选型的规范性方法。第三，论文中的项目和生态状态高度时间敏感，需要持续更新。

D.5 个人理解模板

我认为本文最有启发的地方是把 agent 看作闭环控制系统。模型只是控制器中的一个决策部件，真正决定系统行为的是模型、上下文、工具、执行环境、评测和治理的耦合结果。因此，提升 agent 可靠性不能只依赖更强模型，还需要系统性改进 harness。

附录 E：ETCLOVG 七层对照表

层	关注问题	典型组件	常见失败	需要的证据	---	---	---	---	---	E	动作在哪里执行	容器、microVM、浏览器、桌面 VM、OS 权限沙箱	逃逸、环境不可复现、依赖坏、启动慢	sandbox config、环境快照、资源日志	T	如何调用能力	MCP、A2A、function calling、OpenAPI、tool registry	选错工具、schema 错、返回污染上下文	工具列表、schema 版本、调用参数、返回	C	模型看见什么	prompt、history、retrieval、memory、compaction	忘约束、context rot、context drift、缓存失效	上下文快照、检索记录、压缩摘要	L	任务如何推进	loop、state machine、task graph、checkpoint、handoff	循环、过早停止、状态丢失、交接失败	task state、checkpoint、artifact、事件日志	O	如何观察系统	trace、metrics、cost dashboard、anomaly detector	无法复现、无法归因、成本失控	span tree、token/cost/latency、错误日志	V	如何判断质量	benchmark、grader、rollout、regression suite	分数噪声、grader 偏差、只看 outcome	rollout trace、grader 版本、失败归因	G	如何约束行为	permission、identity、policy、hook、audit	越权、泄露、prompt injection、审计缺失	policy decision、身份、批准记录、audit log
---	------	------	------	-------	-----	-----	-----	-----	-----	---	---------	------------------------------	-------------------	--------------------------	---	--------	--	-----------------------	------------------------	---	--------	--	------------------------------------	-----------------	---	--------	--	-------------------	-------------------------------------	---	--------	---	----------------	-----------------------------------	---	--------	---	---------------------------	------------------------------	---	--------	---------------------------------------	-----------------------------	-----------------------------------

附录 F：如果要自己设计一个 agent harness

这一节把论文转成工程 checklist。它不是原论文的逐字内容，而是基于 ETCLOVG 的实践化整理。

第一步，明确任务和威胁模型。你要做的是 coding agent、browser agent、research agent、enterprise workflow agent，还是 multi-agent platform？它会不会接触敏感文件、凭证、真实用户账号、生产数据库？如果只是本地代码修改，OS 权限沙箱可能够用；如果要运行不可信代码，microVM 更合适。

第二步，定义工具边界。不要一开始暴露全部工具。先列出核心动作：读文件、写文件、运行测试、搜索仓库、访问网页、调用 API。为每个工具写 agent-facing 描述，设计清楚参数和错误返回。工具描述要可版本化。

第三步，设计上下文策略。决定系统提示、工具 schema、项目说明、历史消息、检索结果、长期记忆的优先级。为长程任务准备 progress file 或 notes file。规定何时 compaction、何时清理工具结果、何时启动子 agent。

第四步，设计生命周期。任务从哪里进入？如何拆分？如何记录进度？失败后从哪里恢复？如何判断完成？是否要生成 PR、报告或 artifact？人类在哪些节点 review？

第五步，接入观测。至少记录每次模型调用、工具调用、上下文构造、文件 diff、测试结果、token/cost/latency、错误和重试。没有这些，后续无法优化。

第六步，建立评测。先做小型 deterministic tests，再做完整 rollout。评测不只看成功率，还看成本、延迟、轨迹质量和策略合规。每次 harness 变更都应跑回归。

第七步，加治理。设置文件和网络权限、命令 allow/deny list、凭证 vault、人工确认点、audit log。高风险工具调用前必须检查 policy。审计记录要能回答事后责任问题。

第八步，持续简化。随着模型或任务变化，定期消融 harness 组件：某个子 agent 是否仍必要？某个 context reset 是否降低质量？某个 guardrail 是否误报太多？可靠 harness 不一定越来越复杂，而应该越来越合适。

附录 G：与其他综述的区别

很多 agent 综述按能力模块组织，例如 planning、memory、tool use、multi-agent、safety。这类综述适理解模型能做什么。本文不同，它按系统工程层组织，适理解怎样把模型部署成可靠 agent。

与 memory survey 相比，本文不只讨论如何存取记忆，还讨论记忆如何影响执行、评测和治理。与 tool-use survey 相比，本文不只讨论工具调用能力，还讨论工具协议、工具发现、session 状态和工具安全。与 agent benchmark survey 相比，本文不只讨论数据集和分数，还讨论执行环境 readiness、trace-native evaluation 和 regression feedback。与 safety survey 相比，本文把安全放进 governance 层，与权限、身份、审计和组织责任连接。

因此，本文更像 agent 系统的“架构综述”，而不是单个算法方向综述。它对入门者有用，是因为它告诉你一个 production agent 需要哪些系统部件；对研究者有用，是因为它指出了很多跨层问题仍缺少严格定义和 benchmark。

附录 H：一页式复习提纲

核心命题：长程 LLM agent 的可靠性由模型和 harness 共同决定，harness 经常是现实瓶颈。

三阶段演进：Prompt engineering -> Context engineering -> Harness engineering。

七层框架：

- E：执行环境与沙箱，解决安全、复现和活性。
- T：工具接口与协议，解决能力暴露和动作空间控制。
- C：上下文与记忆，解决模型每步看什么和长期状态。
- L：生命周期与编排，解决任务如何跨调用推进和恢复。
- O：可观测与运维，解决 trace、成本、失败诊断。
- V：验证与评测，解决 benchmark、rollout、归因和回归。
- G：治理与安全，解决权限、身份、策略、审计和人类责任。

三大跨层矛盾：

- 成本-质量-速度三难。
- 能力-控制权衡。
- harness 耦合问题。

五个开放问题：

- 如何加固和扩展执行环境。
- 如何维护长程可靠状态。
- 如何从 trace 诊断失败。

- 如何标准化 agent/tool/human handoff。
- 如何随模型变强持续简化 harness。

一句话结论：未来 agent 工程的重点不是只写更好的 prompt，而是构建可执行、可观测、可验证、可治理的长程控制系统。